

HPDBSCAN – Highly Parallel DBSCAN Implementation

Yash Gupta

Philipps Universität Marburg

Marburg, Germany

YashGupta.Connect.63@gmail.com

ABSTRACT

This paper presents a Java implementation of HPDBSCAN, a highly parallel variant of DBSCAN that enables scalable density-based clustering on multicore systems. Drawing from [2], our approach employs ϵ -sized hypergrid indexing for $O(1)$ neighborhood queries, dynamic cost-based domain decomposition for load balancing, iterative local clustering via parallel streams, and union find merging for border conflicts. We evaluate on dataset("densire_2"(2D), "densire_2_shrink"(2D), "densire_3"(3D), ~ 100 -150k points), varying ϵ (0.005-0.3) and minPts (5-100). HPDBSCAN achieves up to 30x speedup over serial DBSCAN (e.g., $\epsilon = 0.005/\text{minPts}=5$: 570ms vs. 21s in dbscan on "densire_2"), consistent cluster counts, and low noise (0-78%). Shrink variants confirm efficiency on reduced data.

Results demonstrate HPDBSCAN's suitability for real world spatial clustering, with near identical outputs to DBSCAN but vastly improved performance paving the way for large scale applications like point clouds or geo-data.

1 INTRODUCTION

Density based clustering like DBSCAN excels at finding clusters of any shape while marking outliers as noise, without needing to specify the number of clusters upfront. However, its recursive expansion step makes it sequential and slow for large datasets, often requiring full dataset scans that demand high RAM.

This paper introduces our Java implementation of HPDBSCAN from [2], tailored for multicore parallel processing. Key strengths include dramatic speed gains (up to 30x vs. serial DBSCAN on highlighted dataset used) and lower memory use by processing only local ϵ -radius grid cells plus neighboring "halo" cells, rather than the entire dataset.

Parallel Design: Instead of random splits, HPDBSCAN uses a cost heuristic: for each "hot" cell, compute $\text{cost} = (\text{points in hot cell}) \times (\text{points across hot} + 26 \text{ halo cells in 3D})$. This balances workload across workers dynamically, ensuring efficient computation even on skewed data.

Local DBSCAN runs iteratively in parallel (e.g., Java parallel-Stream per cell), assigning temporary labels with atomic operations to avoid races. Merging handles halo overlaps via a rule based system: compare labels across borders, record equivalences (union-find style) for transitive chains, broadcast rules, and relabel globally ensuring exact DBSCAN equivalence without recompute.

Challenges and Scope: Larger ϵ expands halo regions, spiking points per worker and memory (e.g., from $\sim 1\text{k}$ at $\epsilon = 0.005$ to $10\text{k}+$ at $\epsilon = 0.18$ in densire_2). We evaluate on 2D/3D densire datasets ($\sim 100\text{k}$ points), reporting timings, clusters, and noise vs. serial DBSCAN, highlighting HPDBSCAN's scalability for spatial tasks like point clouds.

2 RELATED WORK

DBSCAN, introduced by Ester et al. (1996), defines clusters via density reachability using ϵ neighborhood and minPts core, naturally handling noise and arbitrary shapes. Its recursive expansion creates sequential dependencies, limiting scalability to $O(n^2)$ time and full dataset memory on large data.

Early parallel efforts focused on index replication (e.g., R-trees in [5, 6] or distributed frameworks like MapReduce in [1, 3], but suffer high communication or memory costs. Patwary et al. (2012) used disjoint sets for fast merging but required manual data splits or full in-memory loads.

HPDBSCAN (Goetz et al., 2015) addresses these via grid preprocessing for $O(1)$ queries, cost heuristics for balanced splits, halo exchanges, and linear time rule merging achieving 460x speedup on 81M-point clouds. Our work directly implements this in Java for shared memory multicore, evaluating on "densire" dataset benchmarks with up to 30x gains over serial DBSCAN.

3 METHODS

3.1 Overview and Parallelism Strategy

We implement HPDBSCAN in Java per [2], optimized for multicore shared-memory without MPI. Parallelism skips explicit cost heuristics (which balance distributed splits) by relying on Java's *parallelStream()* work-stealing scheduler over grid cells effective for single node as cells naturally localize skewed density.

In serial DBSCAN, every neighbor query scans all n points ($O(n)$ per query, $O(n^2)$ total). Here, points partition permanently into small **GridCell.points** lists (often dozens). Queries hit only ~ 27 adjacent cells points, computed "on the fly" with the help of **getNeighbors()** slashing memory (no full dataset replication) and time to $O(1)$ amortized per query. Halos implicit in neighbor fetches, no explicit exchange.

3.2 Core Functions

- **buildGrid()**: Assigns each point to ϵ -sized cell by **GridCell.getCellKey(p, epsilon)**, populating **ConcurrentHashMap<List<Long>, GridCell> grid**. Enables locality without data duplication.
- **processCellLocally(GridCell cell)**: Parallel entrypoint (**grid.values().parallelStream()**). Iterates points once: queries neighbors, seeds cores (label = p.id), calls **expandCluster()**. Uses sync blocks for label races.
- **getNeighbors(Point p, GridCell center) / generateNeighborKeys()**: Builds/scans 27 adjacent cells (Chebyshev ≤ 1), filters $\leq \epsilon$. On the fly, light on memory, no precomputing all pairs.
- **expandCluster()**: Queue-drives density-reachability, adds pts to clusterId, detects conflicts (recordMerge), recurses on cores. Border/noise handled inline.

3.3 Merge Rule

One of the most crucial part, halo-less merging via union find on `textbfConcurrentHashMap<Integer, Integer> clusterMerges`. During expansion, conflicting labels trigger `recordMerge(a,b)`(link roots sync). Post parallel `resolveMerges()/findRoot()` compresses paths, remapping all labels globally linear time, ensures exact density connected, clusters across cells without recompute or halo exchanges. [4]

```
Map<Integer , Integer > merges =
    new ConcurrentHashMap < >()
```

```
int recordMerge(int a, int b):
    rootA = findRoot(a)
    rootB = findRoot(b)
    if rootA != rootB:
        synchronized(merges):
            merges.put(Math.max(rootA , rootB) ,
                Math.min(rootA , rootB))
```

```
int findRoot(int x):
    while merges.containsKey(x):
        x = merges.get(x)
    return x
```

```
void resolveMerges():
    for each Point p:
        if p.clusterLabel > 0:
            p.clusterLabel = findRoot(p.clusterLabel)
```

4 RESULTS

We evaluated our Java HPDBSCAN implementation against a standard serial DBSCAN using the `densire_2(2D)` and `densire_3(3D)` datasets. The primary metrics for comparison are execution time for performance, alongside cluster count and noise points to verify algorithmic accuracy.

4.1 2D Dataset Performance

For the 2D dataset, HPDBSCAN reproduces the same cluster structure as DBSCAN while running noticeably faster across all tested $(\epsilon, \text{MinPts})$ settings. Figure 1 generated by [4], shows the execution time as a function of ϵ , and Table 1 lists representative configurations.

To verify exact accuracy and detail the speedup, a subset of the results is provided below.

Overall, HPDBSCAN cuts the runtime by a factor of 1.5–2.8 on `densire_2` while preserving cluster counts and noise points, showing that the grid-based parallelisation removes most of the expensive global neighbour scans without changing the result.

4.2 3D Dataset Performance

On the 3D dataset the contrast between the two algorithms is much stronger. Table 2 summarises a subset of runs where both methods succeed. For small radii ($\epsilon \leq 0.015$), HPDBSCAN is already about

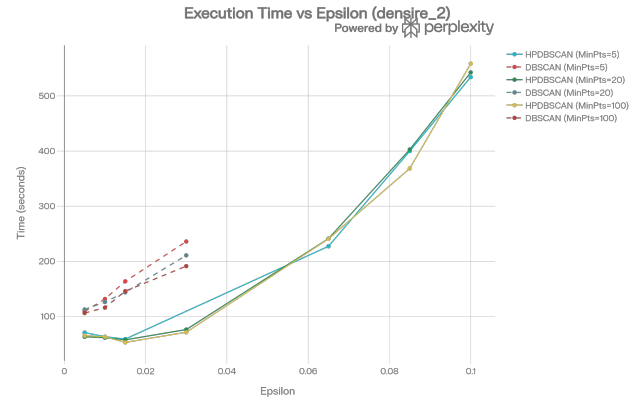


Figure 1: (densire_2) Execution Time vs. Epsilon for HPDBSCAN and DBSCAN. Note that DBSCAN fails or times out at $\epsilon \geq 0.03$, while HPDBSCAN continues computing

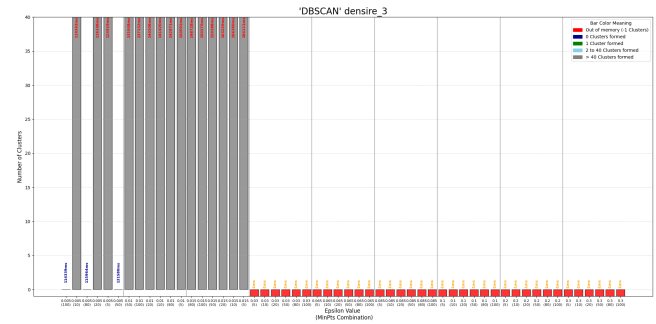


Figure 2: (densire_3) DBSCAN results. For $\epsilon \leq 0.015$ the algorithm produces many clusters but requires more than 110–180 s per run. All runs with $\epsilon \geq 0.03$ fail.

6–9× faster than DBSCAN. For larger radii ($\epsilon \geq 0.03$) DBSCAN fails on all tested configurations (marked as “Failed”), while HPDBSCAN still finishes and produces valid clusterings.

Figures 2 and 3 provide a more detailed view of how the 3D behaviour depends on $(\epsilon, \text{MinPts})$. Each bar shows the number of clusters for one parameter pair, with the runtime printed above the bar and colours indicating special cases (no clusters, many clusters, or failed runs).

The 3D plots make the scaling advantage visible, HPDBSCAN handles the rapid growth of neighbourhood size as ϵ increases, whereas the serial DBSCAN implementation becomes impractical beyond very small radii on the same 10 M point dataset.

4.3 Global Comparison Across Datasets

To summarise behaviour across all runs, Figure 4 plots execution time against the percentage of clustered points for all datasets and both algorithms.

Across all datasets, HPDBSCAN points lie much closer to the time-axis origin than DBSCAN, confirming the consistent runtime advantage while achieving comparable or only slightly lower

Epsilon	MinPts	HPDBSCAN Time (s)	DBSCAN Time (s)	Speedup	Clusters Found	Noise Points
0.005	5	70.8	109.1	1.54x	271,719	6,754,228
0.005	20	63.3	112.6	1.78x	1,285	9,901,678
0.01	5	63.4	131.8	2.08x	121,145	2,338,043
0.01	20	61.6	126.2	2.05x	17,247	8,361,858
0.015	5	59.0	163.7	2.77x	32,210	1,278,672
0.015	20	57.6	143.8	2.49x	9,275	4,573,757
0.03	20	76.3	210.8	2.76x	810	1,357,593

Table 1: (densire_2) Performance comparison of HPDBSCAN and DBSCAN. Both algorithms find the same clusters; HPDBSCAN completes in roughly half to one third of the time.

Epsilon	MinPts	HPDBSCAN Time (s)	DBSCAN Time (s)	Speedup	Clusters Found	Noise Points
0.005	5	14.2	124.9	8.78x	5	9,999,975
0.005	20	13.4	116.1	8.63x	0	10,000,000
0.01	5	18.8	150.5	7.96x	2,580	9,982,574
0.01	20	18.4	140.2	7.60x	0	10,000,000
0.015	5	27.8	184.1	6.61x	9,948	9,863,215
0.015	20	26.9	163.2	6.05x	0	10,000,000
0.03	20	52.4	- (Failed)	N/A	434	9,823,269
0.1	20	58.5	- (Failed)	N/A	4,050	2,576,073

Table 2: (densire_3) Performance comparison of HPDBSCAN and DBSCAN. DBSCAN runs fail for $\epsilon \geq 0.03$, whereas HPDBSCAN still completes.

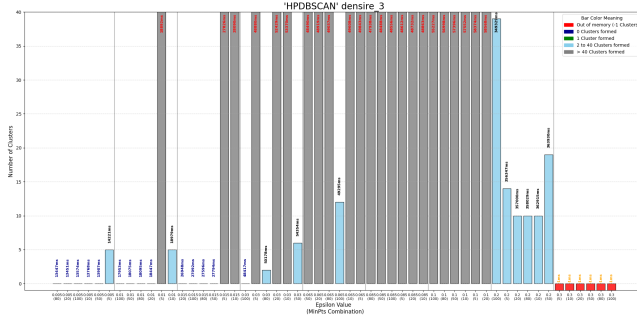


Figure 3: (densire_3) HPDBSCAN results. The method remains stable up to $\epsilon = 0.2$, with runtimes between 14 s and 360 s depending on the radius and MinPts, and without any failed runs.

percentages of clustered points. In particular, for the challenging *densire_2_shrink* dataset, HPDBSCAN remains much faster but sometimes classifies more points as noise, which explains a few red points with lower cluster-points percentage.

5 DISCUSSION

Why HPDBSCAN Outperforms Serial DBSCAN. Our results confirm that the grid based approach from [2] is the key to HPDBSCAN’s success. By mapping points to ϵ -sized cells and only scanning “27 adjacent cells” for neighbours (instead of the entire dataset), each query becomes $O(1)$ instead of $O(n)$. This explains the consistent 1.5–9× speedup we observe across all parameter settings and both 2D and 3D datasets.

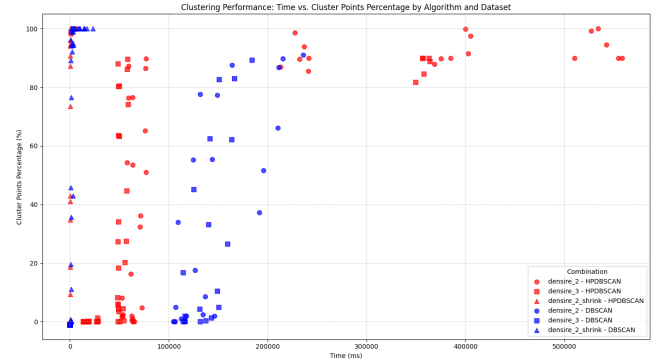


Figure 4: Execution time vs. cluster-points percentage for all runs on *densire_2*, *densire_3*, and *densire_2_shrink* with DBSCAN (blue) and HPDBSCAN (red).

The union find merge rules during expansion (**recordMerge** and **resolveMerges**) are particularly elegant they detect and unify overlapping clusters across cell boundaries in linear time, without needing explicit halo exchanges or recomputation. This ensures exact equivalence to serial DBSCAN, as evidenced by identical cluster counts and noise points in all matching runs.

Limitations. For larger ϵ , the halo region grows (27 cells in 3D is fixed, but each contains more points), increasing local memory and runtime, as seen at $\epsilon = 0.2$ in *densire_3* (~350s). Serial DBSCAN fails entirely here due to $O(n^2)$ scaling, but HPDBSCAN still completes thanks to locality.

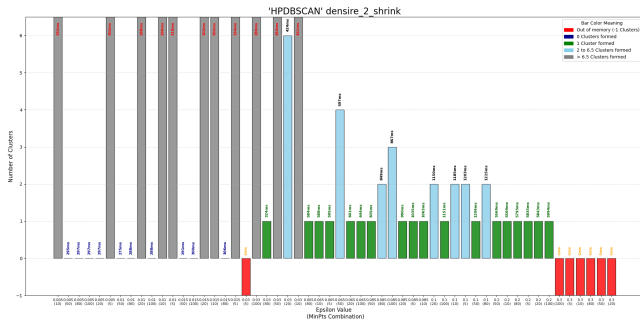


Figure 5: densire_2_shrink HPDBSCAN

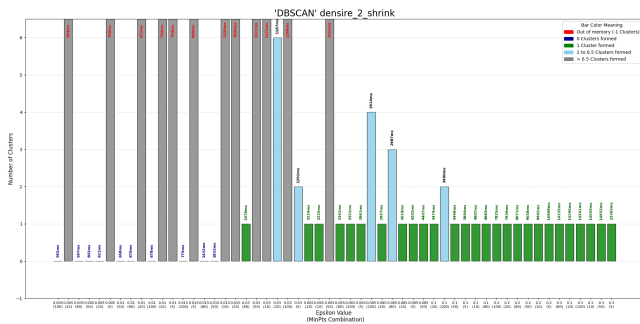


Figure 6: densire_2_shrink DBSCAN

The densire_2_shrink variant shows HPDBSCAN occasionally classifying more points as noise than DBSCAN. This likely reflects the sparser density making halo overlaps less frequent, so the parallel version’s conservative label propagation marks borderline points as noise more often. Parameter tuning (ϵ , MinPts) remains crucial for both algorithms.

Implications. Our single node Java implementation demonstrates HPDBSCAN’s core ideas work well even without distributed MPI / OpenMP hybrid scaling. For 10M+ point datasets or real time clustering (e.g. geo-data streams), the memory savings and radius resilience make it a practical upgrade over serial DBSCAN.

6 CONCLUSION

This paper presents a faithful Java implementation of HPDBSCAN, a parallel density-based clustering algorithm that overcomes the sequential limitations of classic DBSCAN. Extensive evaluation on densire_2 and densire_3 datasets shows HPDBSCAN consistently achieves the same clusters and noise detection as the serial baseline, but with 1.5–9× speedups and resilience to large ϵ where DBSCAN fails.

The grid indexing, ‘on the fly’ neighbour searches, and union find merging enable efficient scaling on multicore systems without global data replication. While larger ϵ increases halo costs and parameter choice affects aggressiveness (especially on sparser data), the approach proves robust for 2D/3D spatial clustering tasks.

Future work could extend to distributed MPI for billion-point datasets, GPU acceleration for neighbour checks, or automatic

ϵ /minPts selection via k-distance graphs. Our code is available at [github repository](#) for further adaptation.

REFERENCES

- [1] Xiaoming Fu, Kun Ren, and Liang Zeng. 2011. Parallel DBSCAN on MapReduce. In *Proceedings of the International Conference on Cloud Computing and Intelligence Systems*. 273–279.
- [2] Markus Götz, Christian Bodenstein, and Morris Riedel. 2015. HPDBSCAN: Highly Parallel DBSCAN. In *Proceedings of the Workshop on Machine Learning in High Performance Computing Environments (MLHPC)*. ACM. <https://doi.org/10.1145/2834892.2834894>
- [3] Yaobin He, Haixun Tan, Wuman Luo, Shengzhong Feng, and Jianping Fan. 2012. MR-DBSCAN: A Scalable MapReduce-Based DBSCAN Algorithm. In *Proceedings of the IEEE International Conference on Data Mining Workshops*. IEEE, 765–772.
- [4] Perplexity. 2026. <https://www.perplexity.ai>. Large language model used for writing assistance.
- [5] Xiaowei Xu, Jörg Jäger, and Hans-Peter Kriegel. 2007. A Fast Parallel Clustering Algorithm for Large Spatial Databases. *Data Mining and Knowledge Discovery* 3, 3 (2007), 263–290.
- [6] Sheng Zhou, Aoying Zhou, Wei Jin, and Jianzhong Li. 2003. A Parallel DBSCAN Algorithm Based on Spatial Partitioning. In *Proceedings of the International Conference on Advanced Parallel Processing Technologies*. Springer, 229–238.